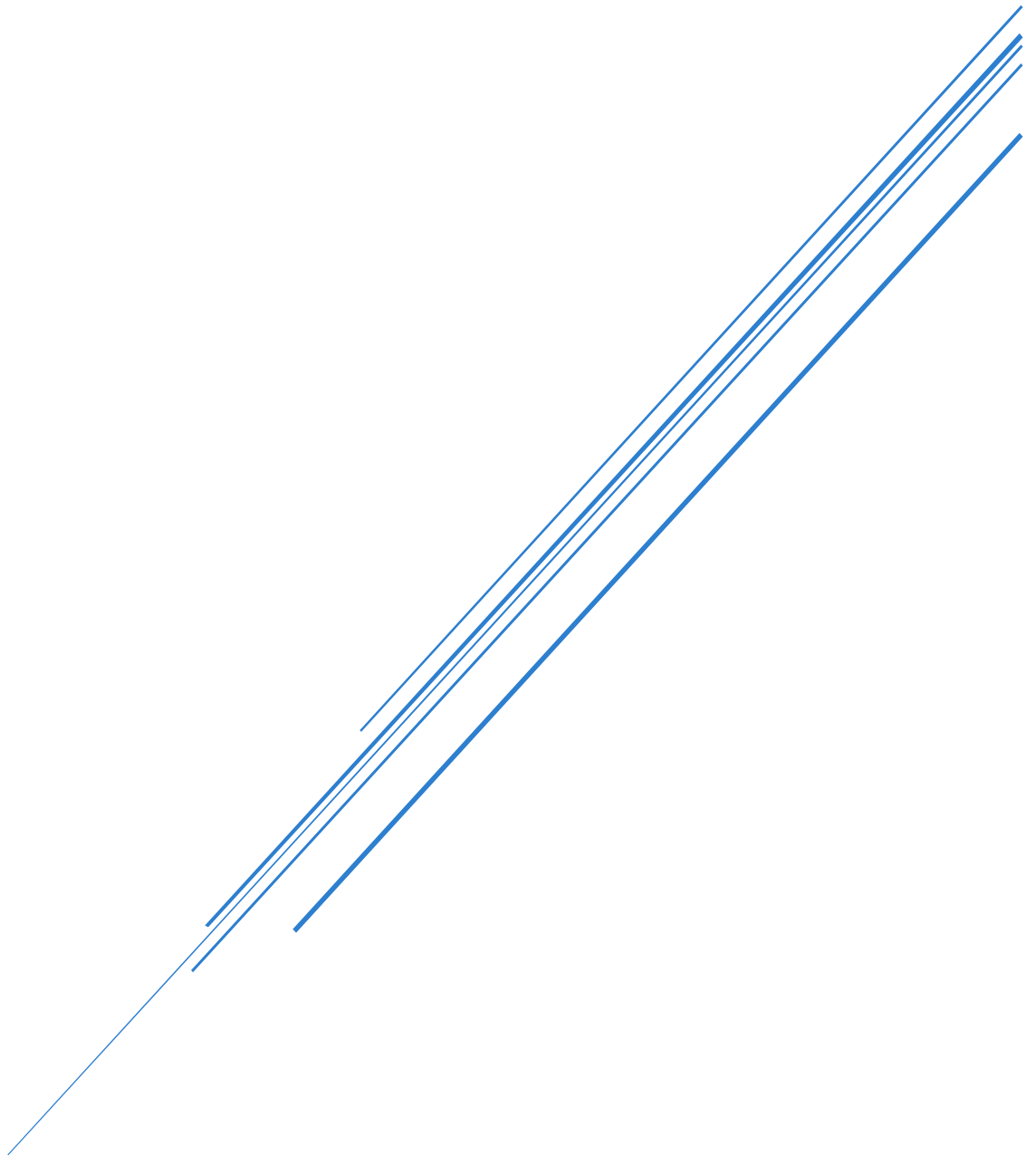


CLOUD DEVELOPMENT A: PART 2

Full Name: Kandyce Smit

Student Number: ST10467645



Lecturer: Farshida Wania
Module Code: CLDV6221

Table of Contents

YouTube and GitHub Link:	2
YouTube Demo Video Link:	2
GitHub Link:.....	2
A. Integrate Local Blob Storage (Azurite).....	3
B. Implementing Error Handling and Validation	5
C. Enhanced Display and Search:.....	7
D. Deployment Verification (Local)	10
Extra Screenshots	11
E. Database Design, Cognitive Search.....	13
▪ Explain how Azure’s Cognitive Search differs from traditional search engines and discuss potential use cases where Cognitive Search would offer a clear advantage.	13
▪ Why is database normalisation important? Discuss the impact of both normalised and denormalised structures on performance and scalability.	19
Code Attribution	26
References	28

YouTube and GitHub Link:

YouTube Demo Video Link:

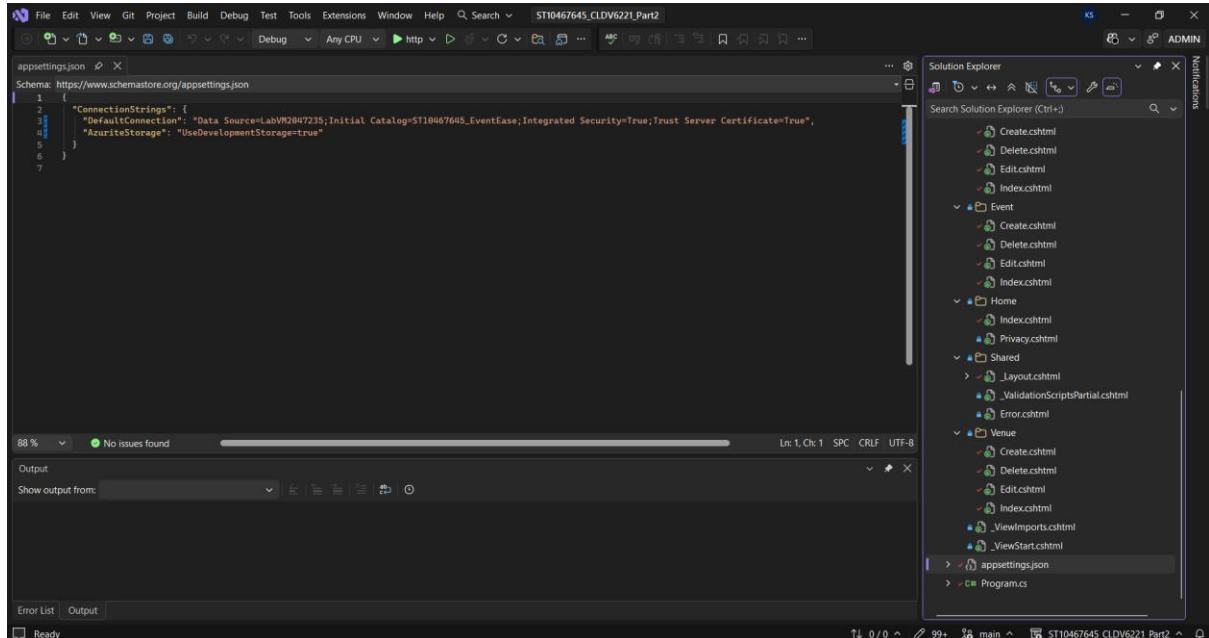
- <https://youtu.be/QaICwTP9h-Q>

GitHub Link:

- <https://github.com/EMGPSD/cldv6211-g1-2026-part2-ST10467645.git>

A. Integrate Local Blob Storage (Azurite)

Screenshot of appsettings.json update:

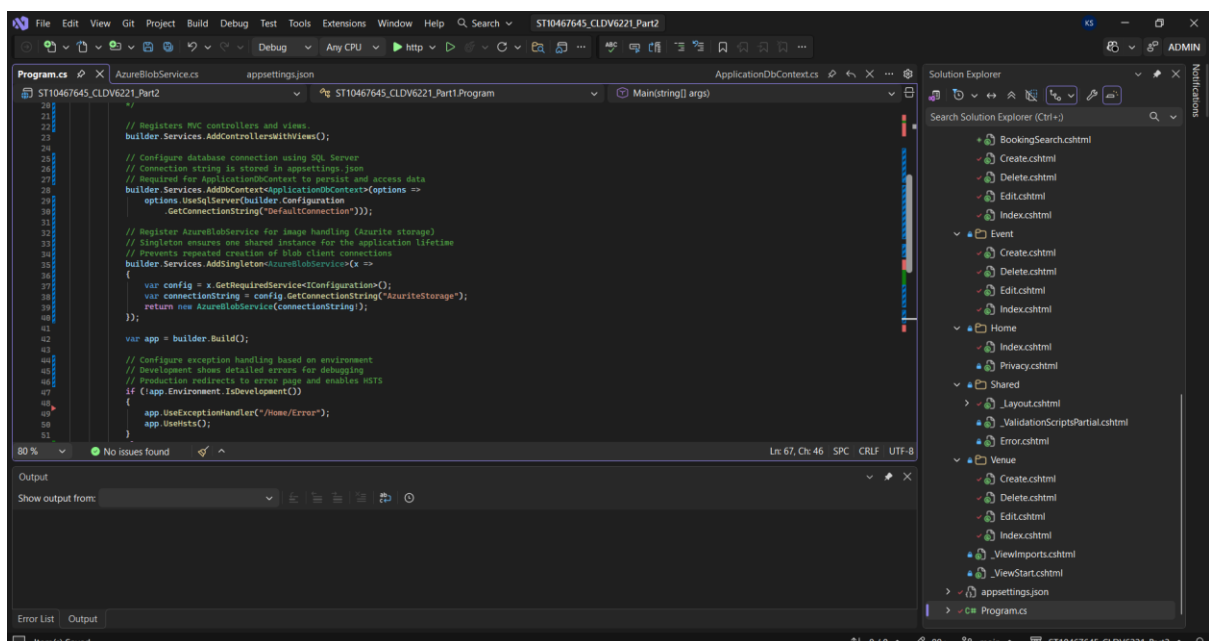


Note: Connection string structure follows guidance from:

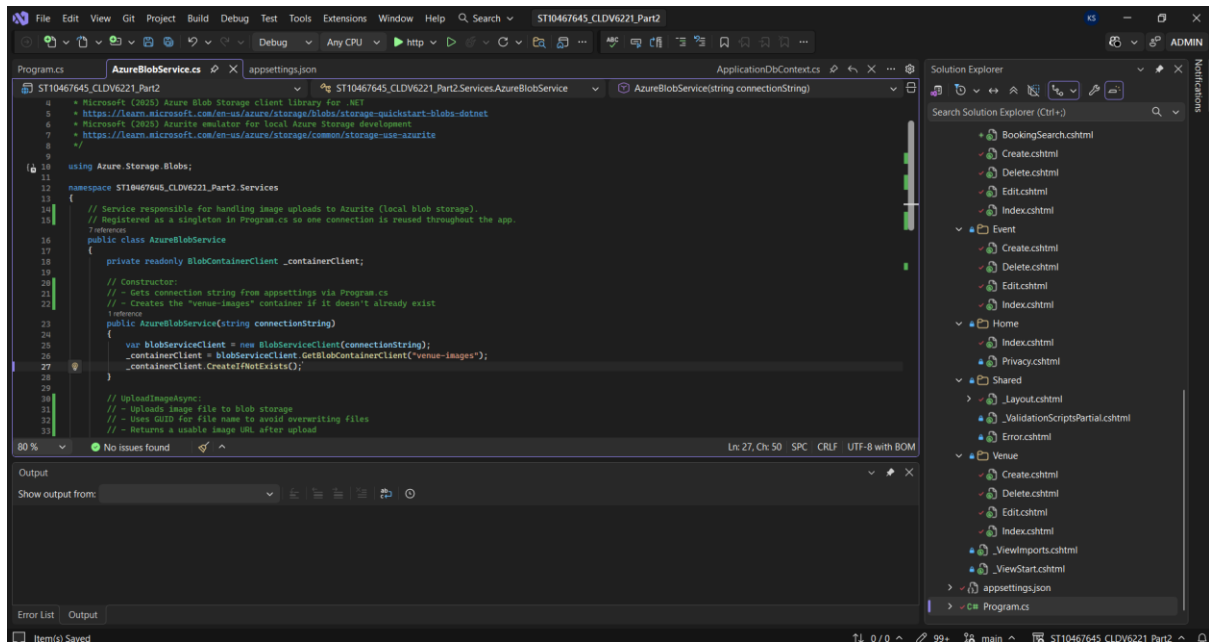
Microsoft (2025) Configuration in ASP.NET Core

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/configuration/>

Screenshot of AzureBlobService registered using Dependency Injection (Singleton) in Program.cs:



Screenshot of Blob container initialization and creation of “venue-images” container:



B. Implementing Error Handling and Validation

Screenshot of double-booking error message on Create Booking form:

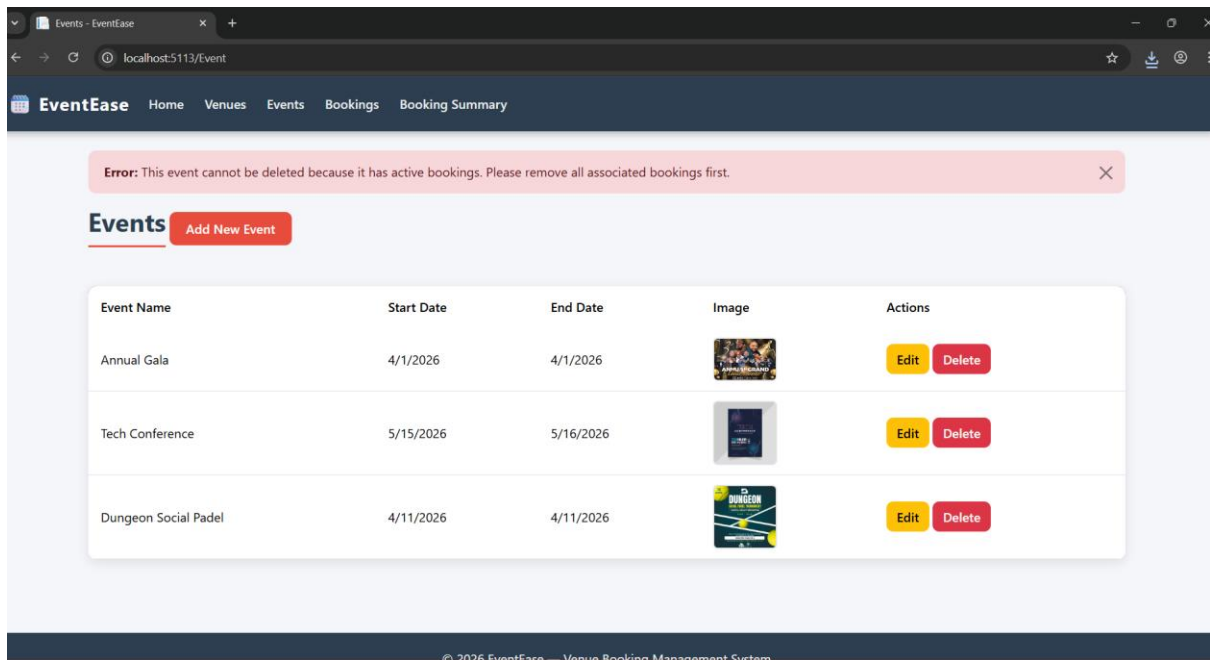
The screenshot shows a web browser window with the URL `localhost:5113/Booking/Create`. The page title is "Add New Booking - EventEase". The navigation bar includes "Home", "Venues", "Events", "Bookings", and "Booking Summary". The main heading is "Add New Booking". A red error message box states: "Please fix the following errors: This venue is already booked on the selected date. Please choose a different date or venue." Below the error message is a form with three fields: "Venue" (a dropdown menu showing "-- Select a Venue --"), "Event" (a dropdown menu showing "-- Select an Event --"), and "Booking Date" (a text input field with a calendar icon and placeholder "mm/dd/yyyy"). At the bottom of the form are two buttons: "Save Booking" (red) and "Cancel" (dark blue).

Screenshot venue delete blocked alert on Venues list:

The screenshot shows a web browser window with the URL `localhost:5113/Venue`. The page title is "Venues - EventEase". The navigation bar includes "Home", "Venues", "Events", "Bookings", and "Booking Summary". A red error message box at the top states: "Error: This venue cannot be deleted because it has active bookings. Please remove all associated bookings first." Below the error message is a table with the following data:

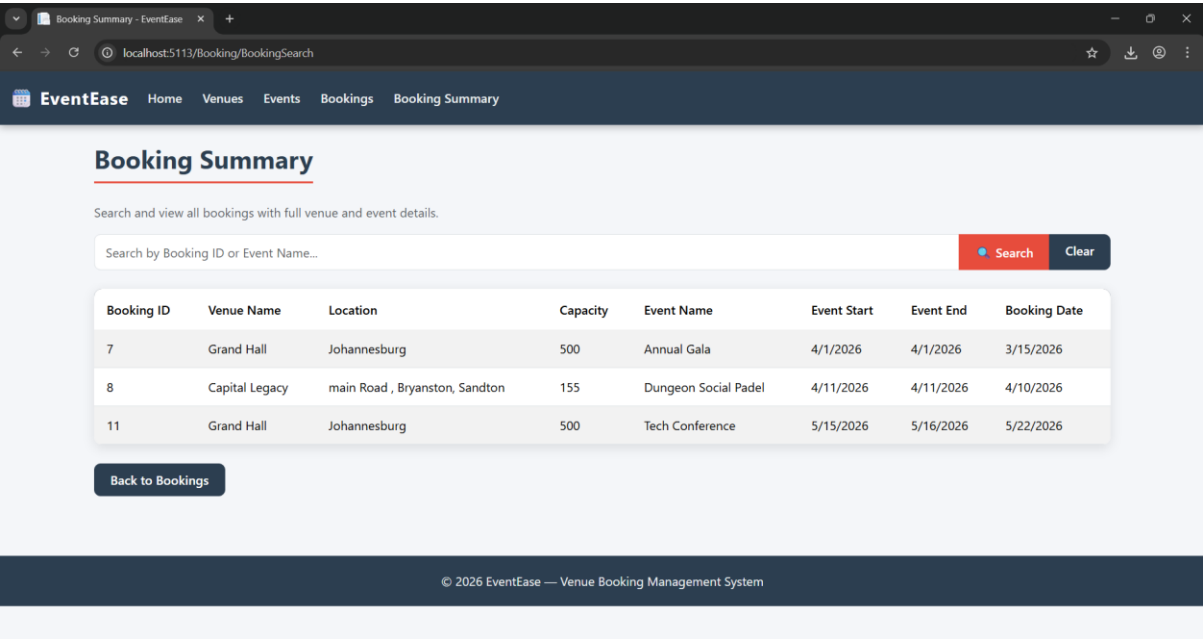
Venue Name	Location	Capacity	Image	Actions
Grand Hall	Johannesburg	500		Edit Delete
Garden Pavilion	Cape Town	200		Edit Delete
Capital Legacy	main Road , Bryanston, Sandton	155		Edit Delete
Kirstenbosch Gardens	Cape Town	350		Edit Delete

Screenshot of event delete blocked alert on Events list:

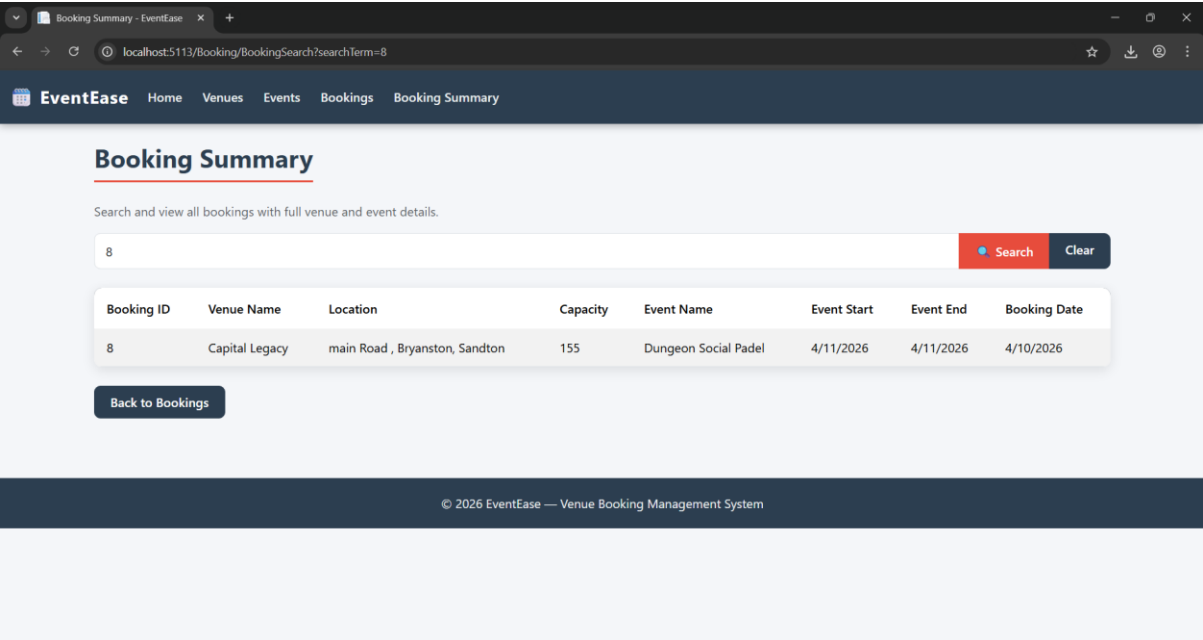


C. Enhanced Display and Search:

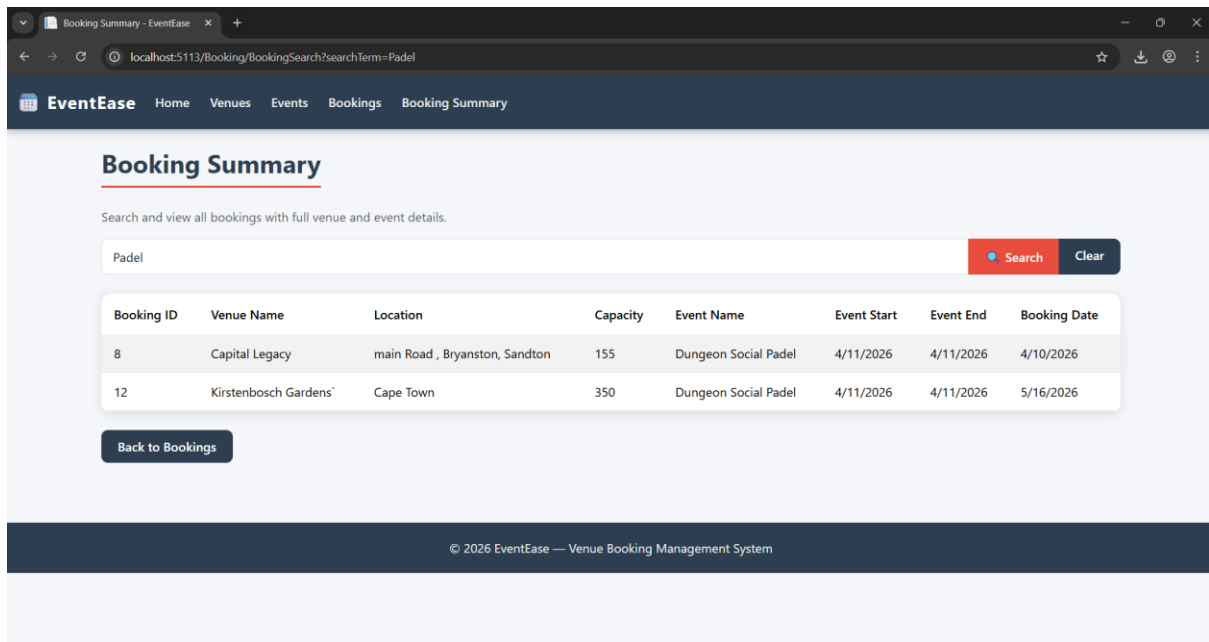
Screenshot of booking Summary page showing full table with venue names and event names:



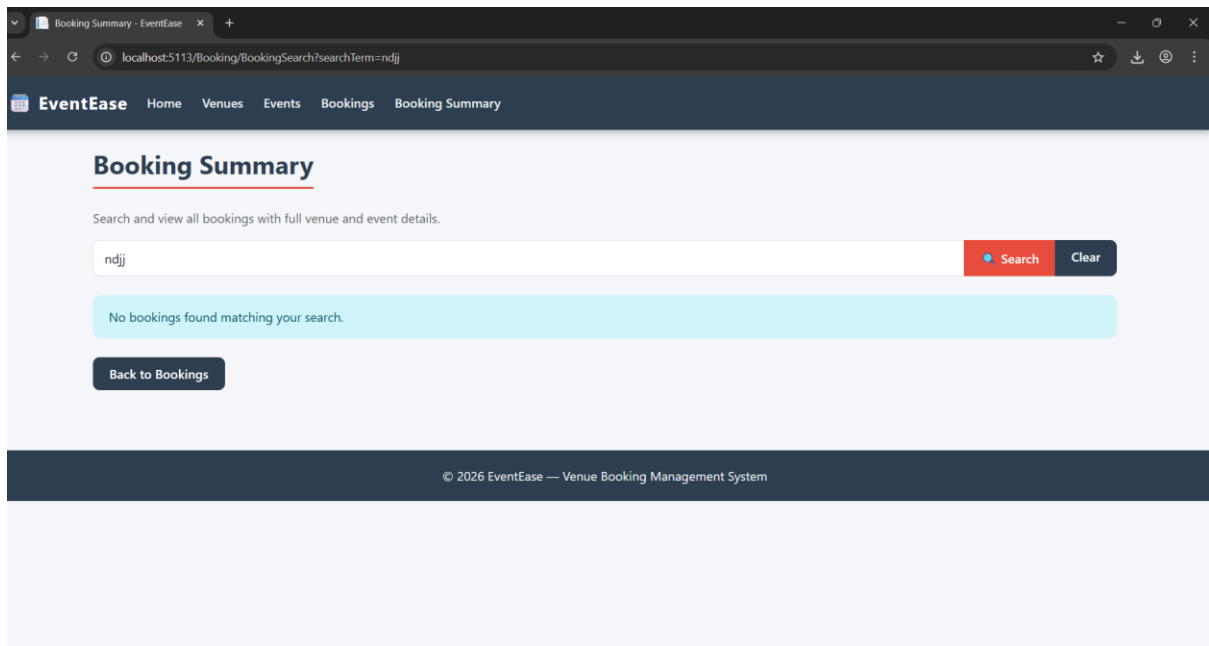
Screenshot of search result filtered by BookingID:



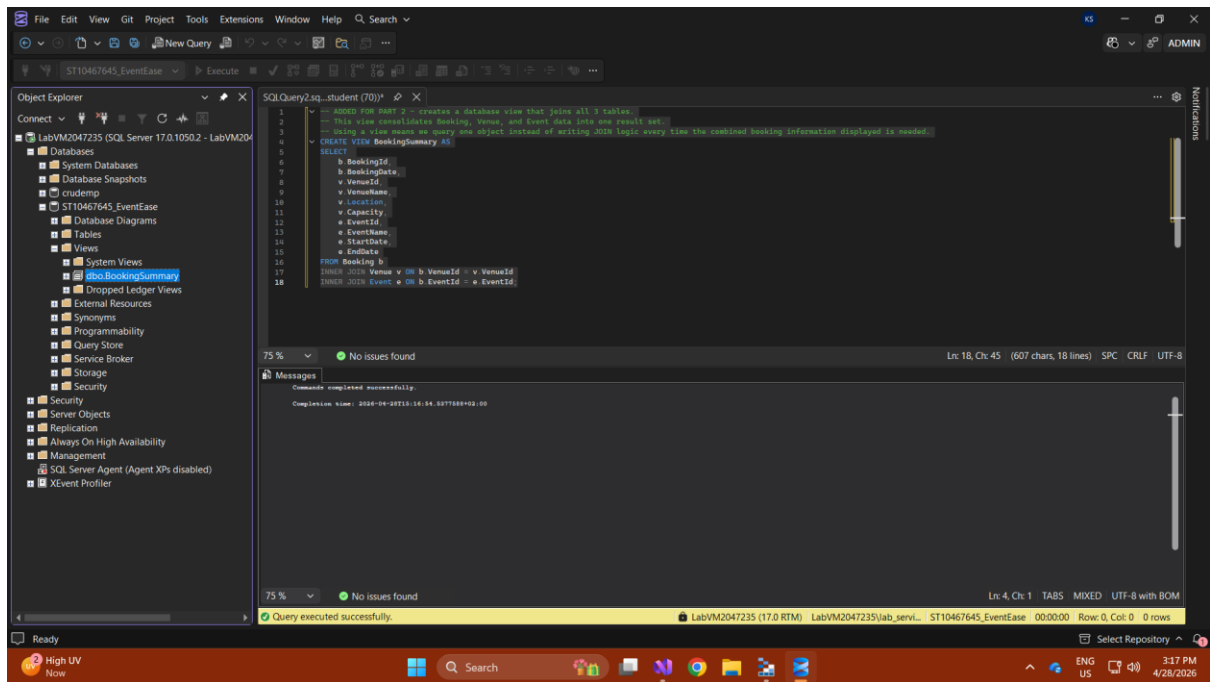
Screenshot of search result filtered by Event Name:



Screenshot of no results message:

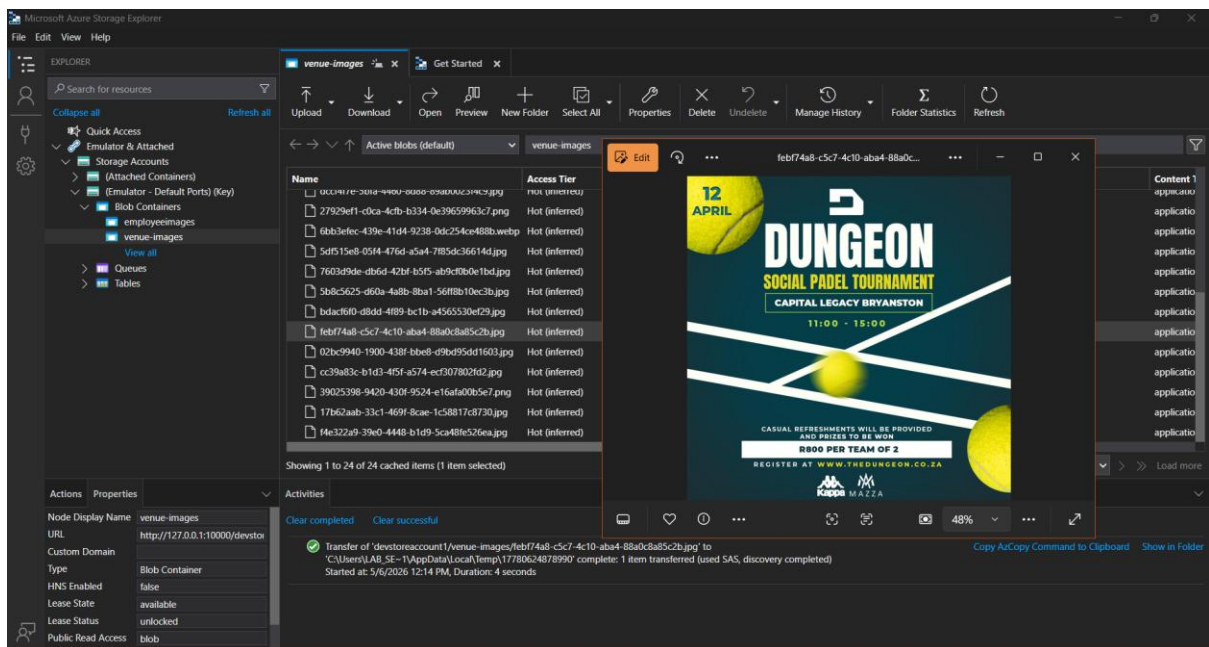
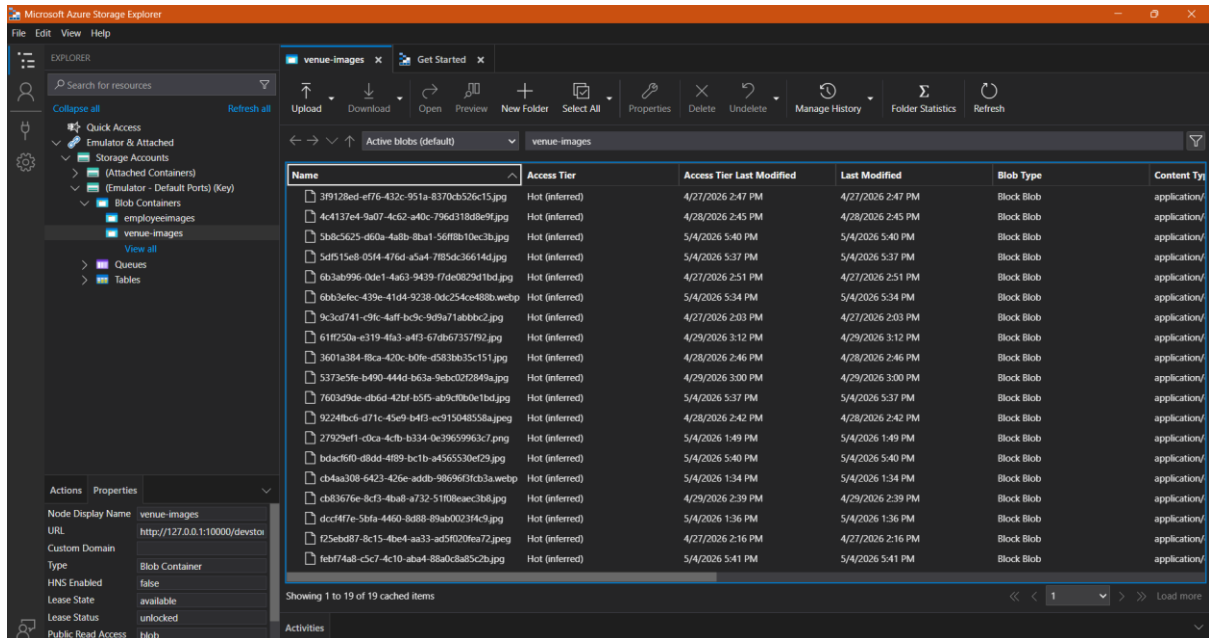


Screenshot of SSMS SQL script for booking summary view:



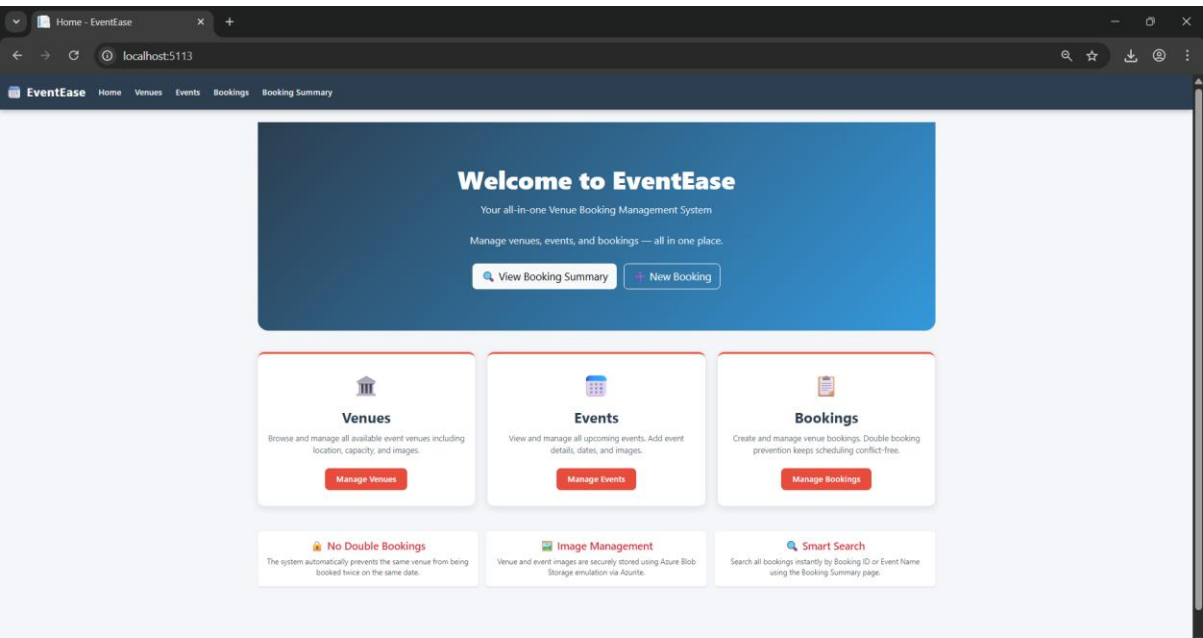
D. Deployment Verification (Local)

Screenshot of Azure Storage Explorer open and right panel showing uploaded blob files listed with their GUID filenames visible:

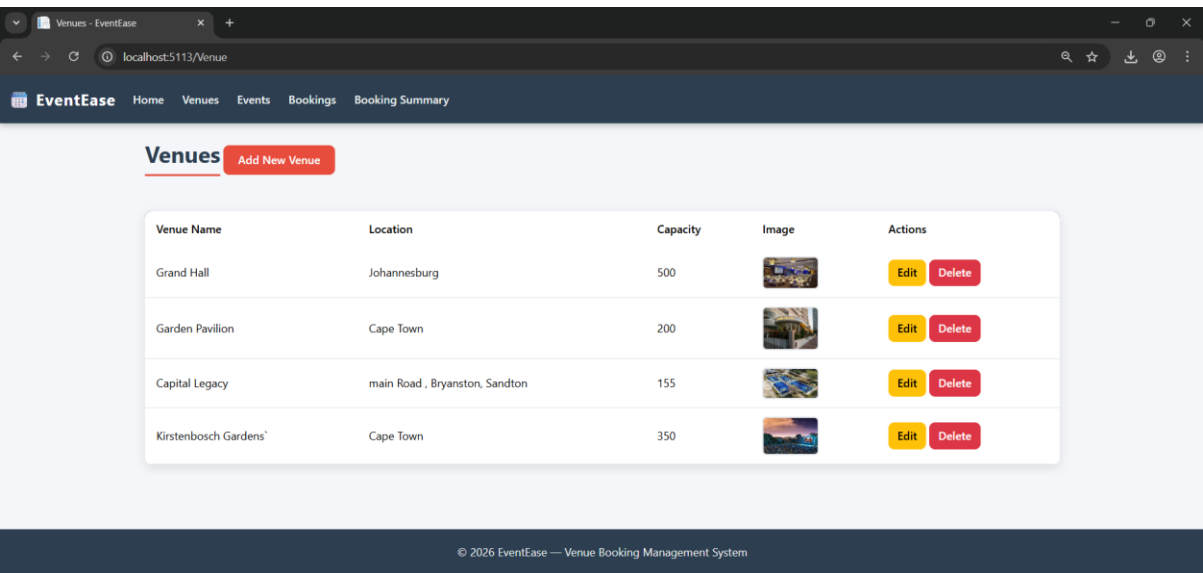


Extra Screenshots

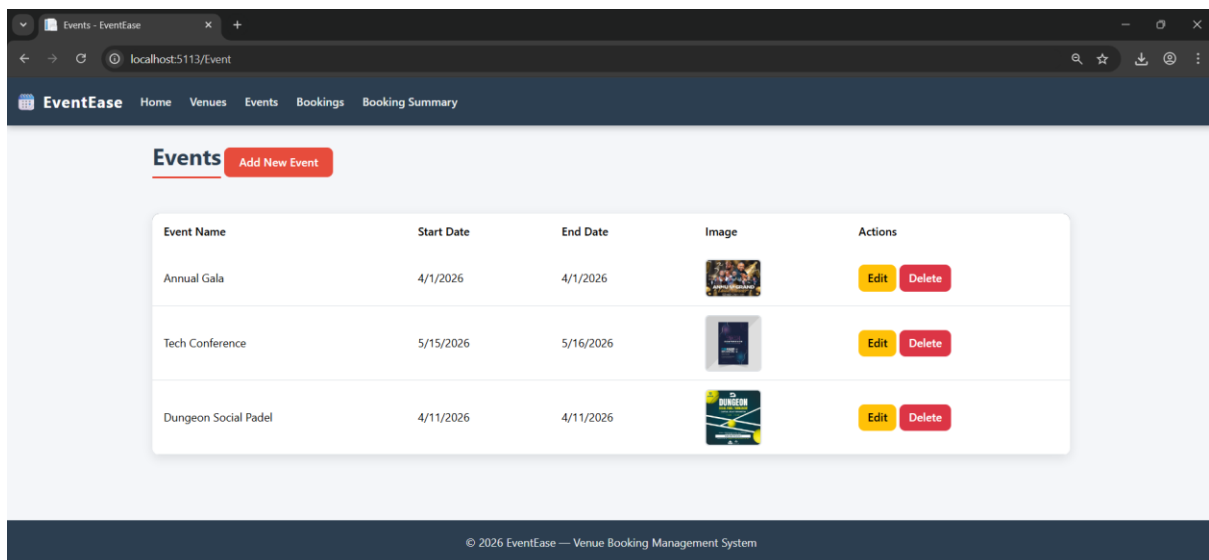
Screenshot of home page:



Screenshot of venues list page:



Screenshot of events list page:



E. Database Design, Cognitive Search

- Explain how Azure's Cognitive Search differs from traditional search engines and discuss potential use cases where Cognitive Search would offer a clear advantage.

What is a Traditional Search Engine?

A traditional search engine uses something called an inverted index. Like the index in the back of a book, an Inverted Index creates a map of every word used in a document to the page number where that word is located. Therefore, when a user searches using a particular term, the engine is searching specifically for that term and provides anything that has been indexed with that term (TechTarget, 2025). Most tools, like SQL, provide a method like "LIKE ' % keyword% '" to perform this function. Most standard website search boxes operate in the same manner.

However, this method of searching is strictly based upon words and not meaning and this is important because:

- No matter how accurately a user enters their term; whether they misspell the term or intentionally enter a misspelling, such as "confreence hall" instead of "conference hall", the search will likely find no applicable results.
- Searches cannot recognise intent. A query entered such as "show me venues near my current location" requires an understanding of both location and context. Keyword matching lacks both.
- It cannot search inside images, PDFs, or scanned documents; therefore, it can only read plain text that is already stored in a field
- The search results returned by keyword matching lacks ranking. Results are presented based upon frequency of occurrence of the searched-for keywords rather than relevance to the searcher (Dorik, 2026).

While these limitations may seem minimal when dealing with smaller amounts of simpler data, as data increases (as is expected with Event Ease), the limitations of traditional keyword searching methods become significant.

What is Azure Cognitive Search?

Azure Cognitive Search, now also known as Azure AI Search, is a different form of implementation altogether. It is a completely managed, cloud-hosted service that ties together your data with artificial intelligence (AI), allowing unified access to your enterprise & web content so that applications can utilise contextual data from multiple sources and provide answers with reliability and relevance (Microsoft , 2025).

Therefore, in simple words, it does not just look for words to match them. It is designed to interpret what the meaning is behind those words.

Azure cognitive search utilises a combination of artificial intelligence, machine learning and natural language processing to determine the intent behind a user's input, rather than simply determining if a user's request contains specific terms. In doing so, instead of asking "Does this record contain those words?" it asks, "What is this person actually trying to find?"

Microsoft has provided an illustration on how Azure Cognitive Search functions:

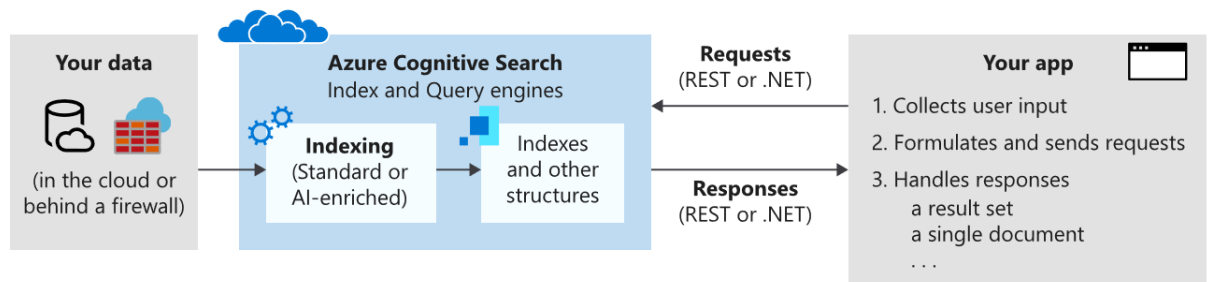


Figure 1: Azure AI Search Architecture (Microsoft , 2025).

As illustrated in Figure 1, Azure cognitive search acts as a mediator between data source(s) and end-users, utilising Artificial Intelligence to enrich and index information prior to allowing users to access said information through searching.

There are several advantages to using Azure cognitive search over traditional search methods:

- Semantic search - it recognises the intended meaning of the query and the context of the query, therefore both "large event space" and "large venue" will result in nearly identical results even though each phrase includes unique wording (GeeksForGeeks, 2025).
- Fuzzy search - it allows for the automatic correction of typos/spelling errors. The fuzzy search function searches for results that are substantially equivalent to the search term. Therefore, a misspelled word will produce search results that include correct spellings (TechTarget , 2022).
- OCR (Optical Character Recognition) - it can identify and search text within scanned images and handwritten documents. When an image is created with text (such as when scanning a document), the Optical Character Recognition process converts the text contained in the image into a machine-readable format. This makes content previously inaccessible due to its form now available for searching (IBM, n.d.).
- Multi-language support - it offers built in translation capabilities and language detection.
- AI-ranked results - search results are presented based upon their relevance to what the user is attempting to locate, not merely based upon the frequency of occurrence of keywords used.
- Document cracking - it can extract and index content from PDF files, Word documents and other file formats that may otherwise not be searchable via traditional search engines.

How Do They Differ?

There are many differences between the two approaches in terms of how search processes work when compared to conventional search techniques. (Dorik, 2026; Microsoft , 2025; Imaginary Cloud, 2025).

Traditional search is reactive meaning it waits for an exact match, whereas cognitive search is intelligent as it understands what user is trying to do.

Traditional search treats every query as a list of words. Cognitive search treats every query as a question with meaning behind it.

Examples of the most apparent differences include:

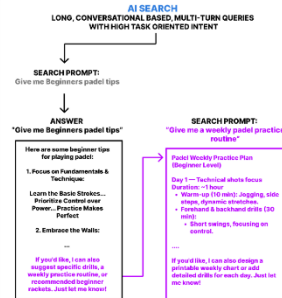
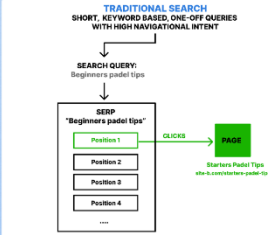
- Traditional search uses exact keyword matching; Cognitive Search uses semantic AI to understand meaning
- Traditional search will typically fail if the user misspells keywords. Cognitive search handles this through fuzzy matching (TechTarget , 2022).
- Traditional search is unable to read images or pdf documents. Cognitive search can read these types of files through optical character recognition (OCR), as well as document cracking to extract additional data (IBM, n.d.).
- Traditional search does not have any form of language intelligence. Cognitive search can process multiple languages without requiring programming changes.
- Traditional search determines the order of search results based upon the number of times a keyword appears. Cognitive search determines the ranking of search results using artificial intelligence-based methods of determining relevance.
- Traditional search often requires manual scaling efforts. Cognitive search scales automatically due to its native integration into Microsoft's azure cloud platform.

A visual comparison between key differences between Traditional and AI-powered search can be seen below:

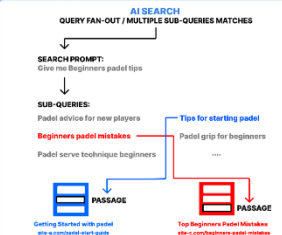
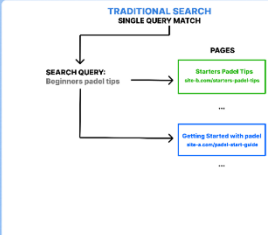
5 Key Traditional vs AI Search Differences

A VISUAL COMPARISON (U2) BY ALEYDA SOLIS

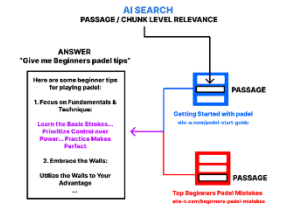
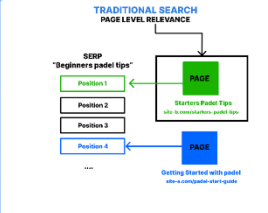
1. SEARCH BEHAVIOR



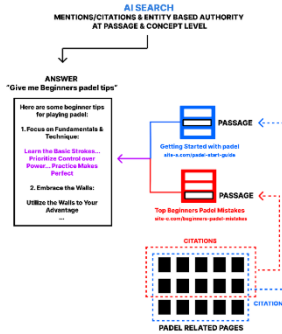
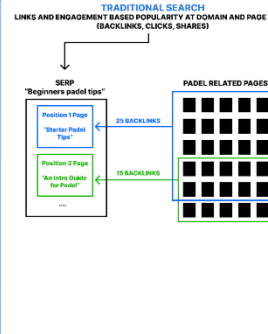
2. QUERY HANDLING



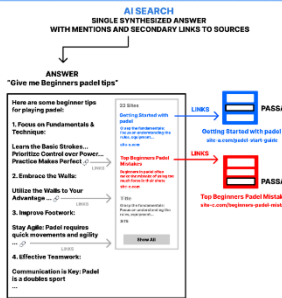
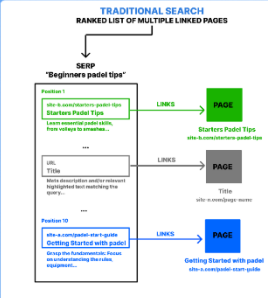
3. OPTIMIZATION TARGET



4. AUTHORITY & POPULARITY SIGNALS



5. RESULTS PRESENTATION



SUBSCRIBE TO THE SEOFOMO NEWSLETTER TO KEEP UP WITH THE LATEST AI SEARCH NEWS

Figure 2: Key Differences Between Traditional and AI-Powered Search (Solis, 2025).

As seen in Figure 2, the gap between traditional and AI-powered search is not just about one feature as it affects every part of how searching works, from understanding the query to delivering ranked, useful results.

Real-World Use Cases Where Cognitive Search Has a Clear Advantage

The benefit of Azure cognitive search shows when you look at practical situations where traditional search would fail or give poor results (Dev.to, 2025; Dev4Side, n.d.; Imaginary Cloud, 2025):

➤ **Use Case 1 - E-Commerce Store**

A customer looks up “red comfortable shoes, size 8”. Cognitive search recognises "comfortable" as an adjective that describes how comfortable they are, which is also related to the overall category of shoe. The user is looking for a shoe that is red, a shoe that is rated high on comfort, and a shoe that comes in their correct size. The search engine will then rank the results from highest to lowest rating for comfort. This means if there is a shoe that is highly rated on comfort but does not come in red, it may still show up in the first few pages of results because of the comfort factor. Traditional search will attempt to match the exact phrase “comfy” and probably bring back nothing or something completely unrelated.

➤ **Use Case 2 - Legal Document Search**

A lawyer wants to pull all contract agreements that reference a termination clause. Cognitive search can read inside pdf files and Microsoft word files to find this. Traditional search cannot open or read pdf or .doc files, therefore, finding information such as this can take forever or even be impossible. (Imaginary Cloud, 2025).

➤ **Use Case 3 - EventEase Venue Booking**

For example, with EventEase, if a booking specialist types “outdoors venue”, “200 people”, “October”. Cognitive search knows that when typing outdoors, you want to know the type of venue; when typing 200 people, you want to know how many people the venue can hold; and when typing October, you want to filter by month. Therefore, cognitive search will return outdoor venues available during October with enough room for 200 guests. In contrast, a simple search box in a traditional system will simply look for each of those individual words within text boxes and return many un-filtered results. (Dev4Side, n.d.).

➤ **Use Case 4 - Hospital Records**

A doctor is searching through a patient’s chart for a particular symptom or medication. Cognitive search can read scanned handwritten patient records and convert them to digital via Optical Character Recognition (OCR). OCR takes images of printed text (and handwritten text) and converts them into usable data that search engines can understand. Therefore, content previously trapped in a physical document or a scanned document is now fully searchable (IBM, n.d.).

An infographic showing where Azure cognitive search stands in comparison to traditional search in terms of these 4 examples:

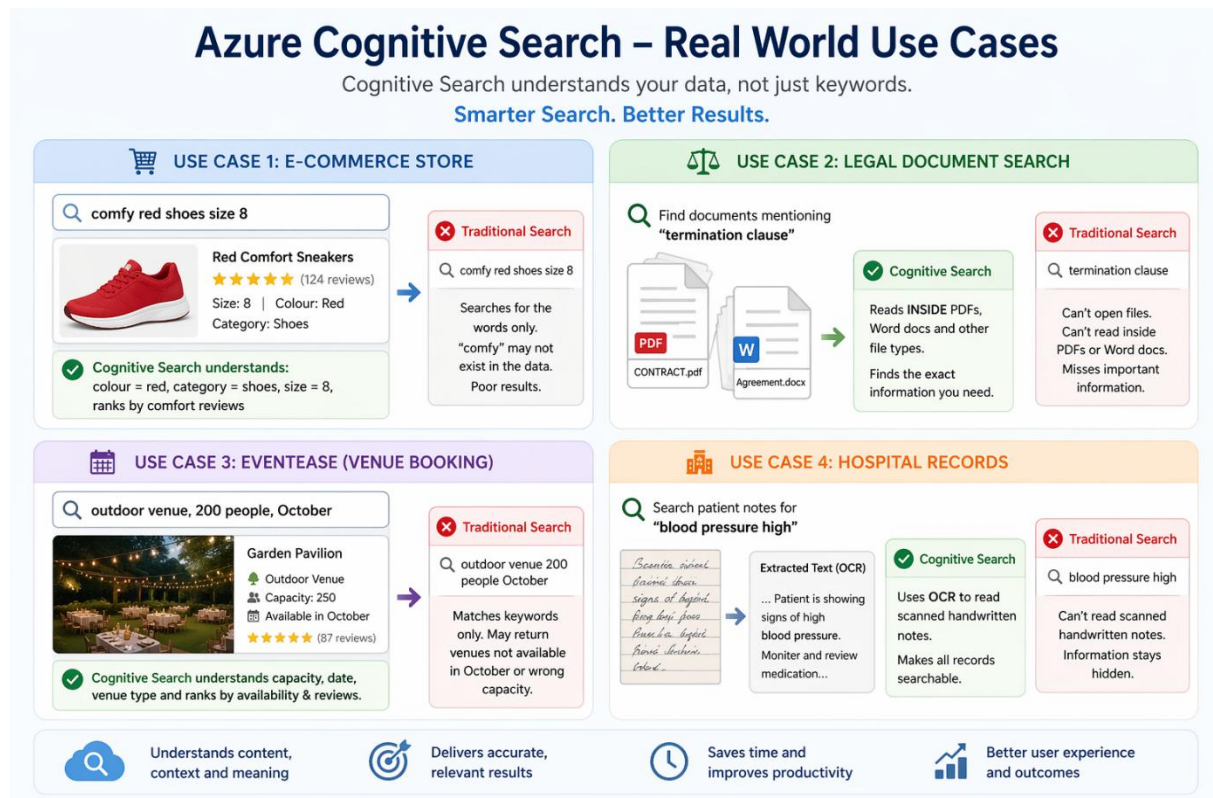


Figure 3: Comparison of Azure Cognitive Search and Traditional Search Across Four Use Cases generated using ChatGPT (OpenAI , 2026).

In each case shown in figure 3, the Azure cognitive search produces significantly better results regarding accuracy, relevance to context, and overall usefulness compared to anything produced by a traditional keyword-based search.

Overall, standard search engines provide an easy way to perform basic keyword-based searches; however, they are not very effective at determining meanings or understanding contexts. Additionally, they do not effectively support different data formats. Azure cognitive search will be able to better understand what the users intend with their queries due to its use of AI. It will also be able to support all types of data formats, for example, images and produce more relevant results. As it is a cloud-based service, it will also automatically scale up as EventEase grows, whereas any standard system that has grown to need additional resources would have had to manually upgrade and/or add new hardware.

- **Why is database normalisation important? Discuss the impact of both normalised and denormalised structures on performance and scalability.**

What is Database Normalisation?

The primary purpose of database normalisation is to ensure a database is structured in such a way that it do not have an excess amount of repeated data in it and to keep all of the information within a database both clean, consistent and easily maintained.

Normalisation is a methodical approach for designing relational table(s) and their relationship(s), which will remove redundancy from your database and reduce storage requirements. Each “normal form” has progressively stricter design rules that are built upon each other. (Google Cloud, n.d.).

Organising a cluttered closet is analogy of normalisation. Rather than stuffing every item in your closet into one large box, you can put items similar to each other in their own labelled boxes so that you can easily locate an item. Similarly, when working with databases, rather than storing all of your data in one massive table, which will contain identical information repeatedly for hundreds of times, you would use multiple, smaller tables, which are related to each other (NDLab, 2025).

Why is it Important?

Without normalisation, databases will rapidly turn into unmanageable messes. There are several reasons why normalising a database is important. They include:

- **Avoiding duplication** - The same piece of data should not be repeated throughout your tables unless necessary. Repeating the same data creates an opportunity for two different versions of the same piece of data to exist in conflict with each other.
- **Easier updates** - When a venue's location is changed, it must be done in one spot. It cannot be done in all the venues' booking records, thus reducing the potential for error when trying to maintain accurate data.
- **Data consistency** - Since data only exists once, changing some pieces of data without changing others does not create inconsistencies within the data base.
- **Storage savings** - Data is only created in one spot, therefore using up less space than would have been needed if it was duplicated across the board. This also saves money by keeping costs low on cloud-based services due to lower usage rates.
- **Scalability** - As the amount of data grows, a normalized database provides a clean, easily managed method to grow from small amounts of data to large amounts of data.
- (DigitalOcean, 2025; TerminalNotes, 2025)

To better understand what happens with poor design, this is an example of poorly designed EventEase booking table which is not normalised:

Booking Table (Not normalised):

- BookingID
- VenueName (repeated for every booking)
- VenueLocation (repeated for every booking)
- EventName (repeated for every booking)
- EventType (repeated for every booking)

Each time an event is booked at for example, "The Grand Hall", EventEase writes out "The Grand Hall" and its location, "Cape Town". If this venue were to move or rename itself, then every single booking record would need to be updated. If one of these updates is incorrect, then the data will become inconsistent. This is a classic example of where normalization helps solve this type of issue. (DEV Community, 2026).

The normalised version splits that into separate, linked tables:

- Venue Table: Which stores venue details once
- Event Table: Stores event details once
- Booking Table: Which links to Venue and Event using foreign keys

Now, after the data is normalised, each piece of information exists in one place, everything else just references it.

The Three Normal Forms

Normalisation occurs in phases. Each phase is referred to as a "normal form". In most cases, achieving the third normal form (3NF) represents a good quality database design (HiSQLBoy, 2025; DigitalOcean, 2025).

Below is an illustration of how an un-normalised table is gradually separated through 1NF, 2NF, and 3NF:

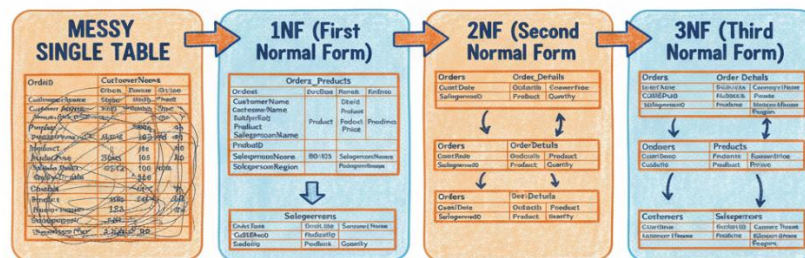


Figure 4: Database Normalisation - Progression from unnormalized to 3NF (HiSQLBoy, 2025).

As shown in figure 4, each normal form expands upon the prior forms resulting in a progressively cleaner and more organised database.

First Normal Form (1NF):

- Every column must consist of a singular atomic value - there may be no lists nor multiple values entered into one cell.
- Every row must be unique
- Example problem: if you store "wedding", "birthday", and "conference" in the same "EventTypes" column as this breaks 1NF as each event type must be in its own row or table.
- (FreeCodeCamp, 2022).

Second Normal Form (2NF):

- Everything from 1NF, plus every non-key column must depend on the entire primary key, not just part of it
- Example problem: if your Booking table has a composite key of BookingID and VenueID, then VenueName should not be in the Booking table as it only depends on VenueID, not the full key, so it belongs in the Venue table
- (Google Cloud, n.d.).

Third Normal Form (3NF):

- Everything from 2NF, plus no column should depend on another non-key column

- Example problem: if VenueCity depends on VenuePostalCode, and VenuePostalCode is not the primary key, then VenueCity needs to be moved into a separate table
- (HiSQLBoy, 2025; DigitalOcean, 2025).

A visual comparison of the rules each normal form enforces and the problems each one solves can be seen in the diagram below:

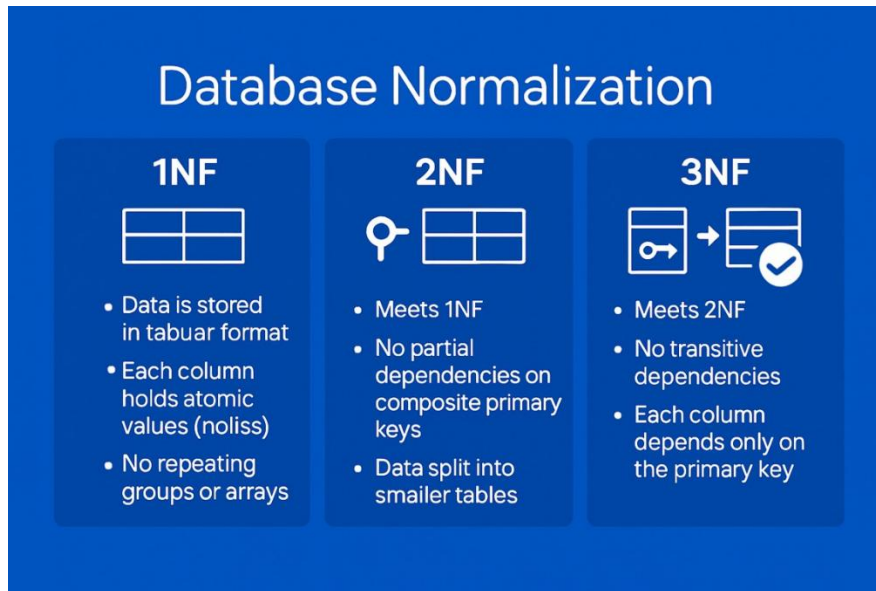


Figure 5: Comparison of the First Three Normal Forms (DigitalOcean, 2025)

Figure 5 shows how normal forms build on previous forms by adding additional rules for refining the Data model.

Normalised vs Denormalised - Performance and Scalability

After understanding what normalisation is; then the subsequent issue becomes if data should always be normalised. The answer is simply that it is determined by your specific use case which leads directly into the concepts of read-heavy vs write-heavy workloads (TechTarget, 2024; SolarWinds, n.d.):

Normalised databases are best when:

- Data is written or updated frequently (write-heavy)
- Accuracy and consistency are the top priority
- The system cannot afford to have duplicate or conflicting records
- Examples: a booking system, a banking app, an inventory management system
- (SolarWinds, n.d.).

Normalising your data means you can write to the database quickly, but reading the data will take longer because you need to do JOINS to get information from different parts of the database. When data is normalised in this way, read operations may become slower due to the need for JOIN queries (Couchbase, 2025).

Denormalised databases are best when:

- Data is read far more often than it is written (read-heavy)
- Speed of retrieval is the top priority over strict data accuracy
- Examples: dashboards, analytics reports, search results pages
- (Pure Storage, 2024)

When you think about normalisation vs. denormalisation in terms of speed, the main trade off is that you'll get fast writes and slow reads when you're normalised, or you'll get slow writes and fast reads when you're denormalised (TechTarget, 2024).

Denormalisation takes the related data and puts it in a few wider tables, making it possible to read all the relevant data at once in one query, rather than having to join multiple queries. With denormalisation, if anything needs to be changed, it typically needs to be changed in several places, increasing the potential for inconsistencies (Exasol, 2026).

A visual diagram that compares how normalised and denormalised structures differ in layout and data organisation can be seen below:

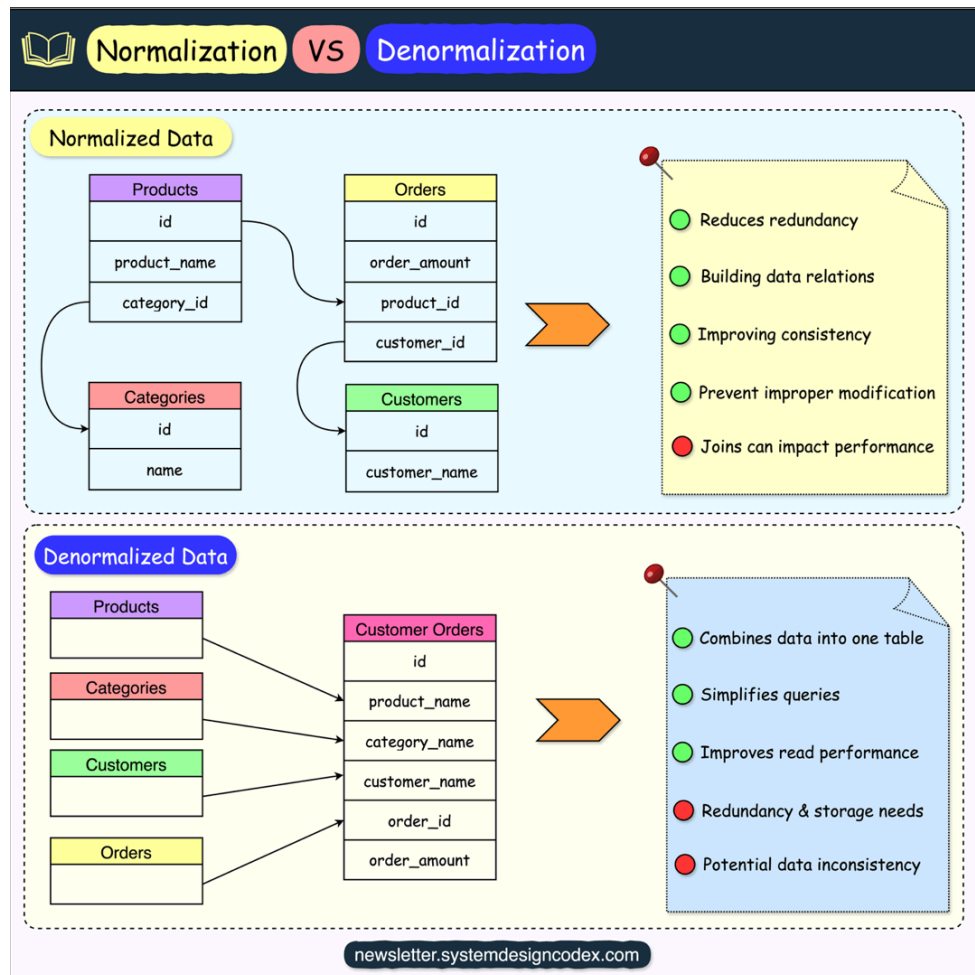


Figure 6: Normalised vs Denormalised Database Structure Comparison (Dashora, 2024)

As seen in Figure 6, the normalised structure distributes the data across related tables, while the denormalised structure brings it all together a table for faster querying.

Why This Matters in the Cloud (In context to EventEase)

It is essential to recognise how these structural differences affect Cloud Environments such as Microsoft Azure. As noted previously, storage costs, query performance, and scalability each directly impact both cost and efficiency in cloud environments (CloudThat, 2024; Netdata, 2025).

Most managed cloud services, including Microsoft Azure SQL Database normally utilise normalized schemas. These schemas are optimised for transactional workloads that require data integrity, reliability, and consistency (SolarWinds, n.d.).

For EventEase specifically:

- Normalisation is the appropriate method to store bookings. Since booking specialists continuously create, edit, and manage bookings, this application has a fundamental need to write. Normalising the database will prevent accidental duplication of venues and/or events. It also allows for one source change of a venue's information to reflect in every place where it was previously listed.
- Denormalisation may be an option for the booking display view. If a booking specialist needs to rapidly review all bookings including venue name(s), event name(s), date(s) and capacity(ies) all on the same screen, a denormalised view, or database view that includes both tables, will allow the page to load faster without sacrificing the integrity of the base data.

Many real-world systems implement a hybrid model as they normalize their primary operational data and then denormalise for specialized high read volume applications such as reporting (Netdata, 2025). EventEase can take advantage of this exact approach as it evolves by maintaining normalised Venue, Event and Booking table structures but create optimised views and/or reporting tables when there is an immediate demand for rapid read performance for items such as dashboard displays or search results.

Relational vs NoSQL Databases

A Relational database organises data in structured tables linked by foreign keys using a fixed schema defined beforehand and includes systems such as SQL Server and azure SQL database. NoSQL databases store data as flexible documents or key-value pairs without a fixed structure, making them suitable for unstructured or rapidly changing data. MongoDB and Azure Cosmos DB are some examples of NoSQL databases (Microsoft, 2026).

The main differences between Relational and NoSQL databases are:

- Relational databases use a fixed schema that must be defined at design time; NoSQL uses a flexible schema that can change over time without restructuring the entire database
- Relational databases scale vertically therefore, more powerful servers); NoSQL scalability is horizontal, meaning, adding more servers, which may be less expensive to implement at large scales
- Relational databases enforce strong consistency ensuring that all data is always accurate; NoSQL supports eventual consistency, where trade-offs exist between accuracy and speed

- Relational databases suit well-structured transactional data; NoSQL better suits high volume data like analytics or real-time feeds
- (SolarWinds, n.d.).

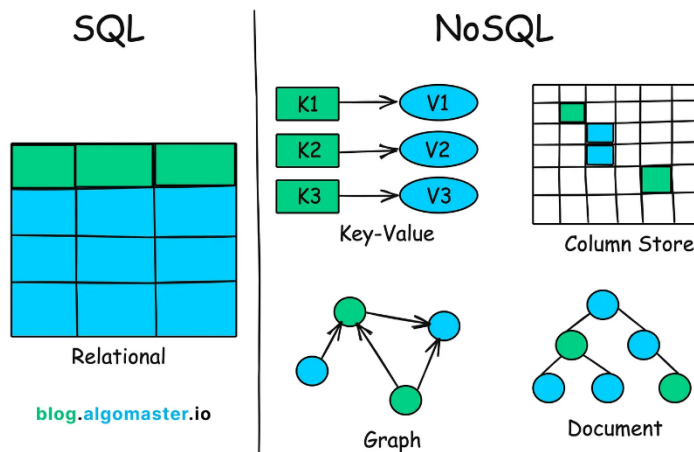


Figure 7: SQL vs NoSQL Key Differences (AlgoMaster, 2024).

As seen in Figure 7, SQL databases organise data in structured table formats whereas NoSQL uses unstructured or semi-structured data in formats that include documents, and key/value pairs which will highlight how they may suit different applications.

Since venues, events, and bookings are highly structured, predictable and relational for EventEase, a relational database is currently the best option. However, once event ease reaches larger scale reporting or analytics requirements, an additional NoSQL solution such as Azure Cosmos DB could be introduced in parallel to handle those read heavy work loads (Microsoft, 2026).

Code Attribution

The following sources were consulted during the development of this application:

Bootstrap, 2025. Bootstrap 5 Documentation. [Online]

Available at: <https://getbootstrap.com/docs/5.0/getting-started/introduction/>

[Accessed 29 April 2026].

Microsoft, 2025. ASP.NET Core MVC Overview. [Online]

Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>

[Accessed 28 April 2026].

Microsoft, 2025. Azure Blob Storage client library for .NET. [Online]

Available at: <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-quickstart-blobs-dotnet>

[Accessed 28 April 2026].

Microsoft, 2025. Azurite emulator for local Azure Storage development. [Online]

Available at: <https://learn.microsoft.com/en-us/azure/storage/common/storage-use-azurite>

[Accessed 28 April 2026].

Microsoft, 2025. Configuration in ASP.NET Core. [Online]

Available at: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/configuration/>

[Accessed 28 April 2026].

Microsoft, 2025. Database views with Entity Framework Core. [Online]

Available at: <https://learn.microsoft.com/en-us/ef/core/modeling/keyless-entity-types>

[Accessed 28 April 2026].

Microsoft, 2025. DbContext Class. [Online]

Available at: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.entityframeworkcore.dbcontext>

[Accessed 28 April 2026].

Microsoft, 2025. Entity Framework Core. [Online]

Available at: <https://learn.microsoft.com/en-us/ef/core/>

[Accessed 28 April 2026].

Microsoft, 2025. Model validation in ASP.NET Core MVC. [Online]

Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation>

[Accessed 28 April 2026].

Microsoft, 2025. Razor syntax reference for ASP.NET Core. [Online]

Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor>

[Accessed 28 April 2026].

Microsoft, 2025. SelectList Class. [Online]

Available at: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.mvc.rendering.selectlist>

[Accessed 28 April 2026].

Microsoft, 2025. Tag Helpers in ASP.NET Core. [Online]
Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/views/tag-helpers/intro>
[Accessed 28 April 2026].

Microsoft, 2025. TempData in ASP.NET Core. [Online]
Available at: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state>
[Accessed 28 April 2026].

References

- AlgoMaster, 2024. *SQL vs NoSQL - 7 Key Differences You Must Know*. [Online]
Available at: <https://blog.algomaster.io/p/sql-vs-nosql-7-key-differences>
[Accessed 24 April 2026].
- CloudThat, 2024. *The Role of Normalization and Denormalization in Data Architecture*. [Online]
Available at: <https://www.cloudthat.com/resources/blog/the-role-of-normalization-and-denormalization-in-data-architecture>
[Accessed 22 April 2026].
- Couchbase, 2025. *Data Normalization vs. Denormalization Comparison*. [Online]
Available at: <https://www.couchbase.com/blog/normalization-vs-denormalization/>
[Accessed 22 April 2026].
- Dashora, S., 2024. *Database Normalization vs Denormalization*. [Online]
Available at: https://www.linkedin.com/posts/saurabh-dashora_database-normalization-vs-denormalization-activity-7180816725725126656-nNK-
[Accessed 22 April 2026].
- DEV Community, 2026. *SQL Normalization vs Denormalization Explained with 1NF, 2NF, and 3NF*. [Online]
Available at: <https://dev.to/davinceleecode/sql-normalization-vs-denormalization-explained-with-1nf-2nf-and-3nf-2gpa>
[Accessed 22 April 2026].
- Dev.to, 2025. *What is Azure AI Search: Core Features and Use-Cases*. [Online]
Available at: https://dev.to/emma_wags_8dd9b74533690da/what-is-azure-ai-search-3f49
[Accessed 21 April 2026].
- Dev4Side, n.d. *Azure Cognitive Search: What it is, features, and costs*. [Online]
Available at: <https://www.dev4side.com/en/blog/azure-cognitive-search>
[Accessed 21 April 2026].
- DigitalOcean, 2025. *Database Normalization: 1NF, 2NF, 3NF & BCNF Examples*. [Online]
Available at: <https://www.digitalocean.com/community/tutorials/database-normalization>
[Accessed 22 April 2026].
- Dorik, 2026. *AI vs Traditional Search Engines: 8 Key Differences*. [Online]
Available at: <https://dorik.com/blog/ai-search-engines-vs-traditional-search-engines>
[Accessed 21 April 2026].
- Exasol, 2026. *Normalization vs Denormalization*. [Online]
Available at: <https://www.exasol.com/hub/database/normalization-vs-denormalization/>
[Accessed 22 April 2026].
- FreeCodeCamp, 2022. *Database Normalization – Normal Forms 1nf 2nf 3nf Table Examples*. [Online]
Available at: <https://www.freecodecamp.org/news/database-normalization-1nf-2nf-3nf-table-examples/>
[Accessed 22 April 2026].

GeeksForGeeks, 2025. *What is Semantic Search?*. [Online]
Available at: <https://www.geeksforgeeks.org/nlp/what-is-semantic-search/>
[Accessed 21 April 2026].

Google Cloud, n.d. *What is database normalization?*. [Online]
Available at: <https://cloud.google.com/discover/what-is-database-normalization>
[Accessed 22 April 2026].

HiSQLBoy, 2025. *Database Normalization Explained: 1NF, 2NF, 3NF*. [Online]
Available at: <https://www.hisqlboy.com/blog/database-normalization-explained>
[Accessed 22 April 2026].

IBM, n.d. *What is optical character recognition (OCR)?*. [Online]
Available at: <https://www.ibm.com/think/topics/optical-character-recognition>
[Accessed 21 April 2026].

Imaginary Cloud, 2025. *Azure AI Search: Benefits, Use Cases and Implementation*. [Online]
Available at: <https://www.imaginarycloud.com/blog/azure-ai-search-enterprise-guide>
[Accessed 21 April 2026].

Microsoft , 2025. *What is Azure AI Search?*. [Online]
Available at: <https://learn.microsoft.com/en-us/azure/search/search-what-is-azure-search?tabs=indexing%2Cquickstarts>
[Accessed 21 April 2026].

Microsoft, 2026. *Azure Cosmos DB - Database for the AI Era*. [Online]
Available at: <https://learn.microsoft.com/en-us/azure/cosmos-db/overview>
[Accessed 24 April 2026].

NDLab, 2025. *Database Design Fundamentals: Understanding Normalization (1NF, 2NF, 3NF)*. [Online]
Available at: <https://ndlab.blog/posts/database-design-normalization-1nf-2nf-3nf>
[Accessed 22 April 2026].

Netdata, 2025. *Normalized vs Denormalized - Choosing The Right Data Model*. [Online]
Available at: <https://www.netdata.cloud/academy/normalized-vs-denormalized/>
[Accessed 22 April 2026].

OpenAI , 2026. *ChatGPT AI generated image*. [Online]
Available at: <https://chat.openai.com>
[Accessed 21 April 2026].

Pure Storage, 2024. *Denormalized vs. Normalized Data*. [Online]
Available at: <https://blog.purestorage.com/purely-technical/denormalized-vs-normalized-data/>
[Accessed 22 April 2026].

SolarWinds, n.d. *The Key Concepts of Database Optimization*. [Online]
Available at: <https://www.solarwinds.com/database-optimization>
[Accessed 24 April 2026].

SolarWinds, n.d. *The Key Differences Between Normalizing and Denormalizing Databases*. [Online]
Available at: <https://www.solarwinds.com/database-optimization/normalize-vs-denormalize->

[database](#)

[Accessed 22 April 2026].

Solis, 2025. *The 5 Key AI Search vs Traditional Search Differences – A Visual Comparison*.

[Online]

Available at: <https://www.aleydasolis.com/en/ai-search/key-traditional-search-ai-comparison/>

[Accessed 21 April 2026].

TechTarget , 2022. *What is a fuzzy search?*. [Online]

Available at: <https://www.techtarget.com/whatis/definition/fuzzy-search>

[Accessed 21 April 2026].

TechTarget, 2024. *What is denormalization and how does it work?*. [Online]

Available at: <https://www.techtarget.com/searchdatamanagement/definition/denormalization>

[Accessed 22 April 2026].

TechTarget, 2025. *GenAI search vs. traditional search engines: How they differ*. [Online]

Available at: <https://www.techtarget.com/WhatIs/feature/GenAI-search-vs-traditional-search-engines-How-they-differ>

[Accessed 21 April 2026].

TerminalNotes, 2025. *Database Normalization in SQL Explained: 1NF, 2NF, 3NF with Examples*.

[Online]

Available at: <https://terminalnotes.com/database-normalization-in-sql-explained-1nf-2nf-3nf-with-examples/>

[Accessed 22 April 2026].